

# Chapter 3

## Race conditions

Two threads that only read shared data never disturb each other. As soon as one of them writes, the result can depend on the exact order in which the threads' instructions happen to interleave — an order nobody controls. This chapter explains why even `counter++` is not safe, how to reason about interleavings, and which guarantees (“happens before”) you can actually rely on.

### 3.1 The lost update

`counter++` looks like one step. The CPU executes three:

```
mov  eax, [counter]    ; load
add  eax, 1            ; modify (in a register - private to the thread)
mov  [counter], eax    ; store
```

If thread A is interrupted (or simply runs on another core) between its load and its store, both threads can load the same old value:

| Thread A       | Thread B       | counter  |
|----------------|----------------|----------|
| load (reads 5) |                | 5        |
|                | load (reads 5) | 5        |
|                | add, store 6   | 6        |
| add, store 6   |                | <b>6</b> |

Two increments, one effect: a *lost update*. Two threads with three instructions each can interleave in  $\binom{6}{3} = 20$  ways; only the 2 in which one thread's three instructions all come before the other's are correct (module 03's `interleave.c` lists all 20). With a million increments per thread, the final count is usually far too small — and different in every run.

### 3.2 Check-then-act

A race needs no `++`. Any sequence “look at shared state, then act on what you saw” breaks if the state changes in between:

```
if (account->balance >= amount)    /* check */
    account->balance -= amount;      /* act: another thread may have
                                     withdrawn in between */
```

The same pattern appears as “if the file does not exist, create it”, “if the cache has no entry, compute and insert it”, “if the pointer is NULL, allocate”. In a kernel the dangerous variant is

*time-of-check to time-of-use* (TOCTOU): the kernel checks a value in user memory, then reads it again to use it — and another user thread changed it in between (chapter 7).

### 3.3 Critical sections and atomicity

A *critical section* is a piece of code that accesses shared state and must not run concurrently with other critical sections on the same state. Making it *atomic* means: every other thread sees either none of its effects or all of them, never an intermediate state. There are three ways to get there (module 03's `three_fixes.c` compares them):

1. **Lock it:** a mutex around the whole check-and-act (chapter 4).
2. **Use an atomic operation:** `atomic_fetch_add` does load–add–store as one indivisible instruction (`lock xadd`). Only works when the whole critical section is a single such operation.
3. **Do not share:** give each thread its own copy and combine afterwards (chapter 2's partition-then-combine).

Invariants are the key to finding critical sections: “the sum of all balances is constant”, “`count` equals the number of elements in the list”. Every code path that temporarily breaks an invariant is a critical section.

### 3.4 Data race vs. race condition

The two words are often mixed up:

**Data race** (a precise, mechanical definition): two threads access the same memory location, at least one access is a write, and the two accesses are not ordered by happens-before (no lock, no atomic, no join between them). In C and C++ a data race is *undefined behaviour* — the compiler may assume it never happens. ThreadSanitizer finds exactly these.

**Race condition** (a semantic property): the program's correctness depends on timing. A program can have a race condition without a data race (every access is locked, but the lock is released between check and act) and a data race without visible harm (a statistics counter that may be a little off — still undefined behaviour).

### 3.5 Happens-before

Which writes of thread A is thread B guaranteed to see? Only those that *happen before* B's read, where happens-before is built from:

- program order within one thread;
- unlocking a mutex happens before the next lock of the same mutex;
- `pthread_create` happens before the first instruction of the new thread; the thread's last instruction happens before `pthread_join` returns;
- a release store to an atomic happens before an acquire load that reads it (chapter 4);
- transitivity: A before B and B before C gives A before C.

Anything else is unordered, and without an ordering there is no guarantee at all — not even that B ever sees A's write. The compiler may keep a variable in a register for the whole loop:

```
while (!done) {}           /* with a plain `int done`, -O2 may turn this into
                           `if (!done) for (;;)`, - it never re-reads it */
```

and the CPU may make stores visible to other cores in a different order than the program wrote them (chapter 4). `volatile` stops the compiler from caching, but orders nothing between threads; use atomics or locks.

### 3.6 Why testing is not enough

A race that loses one update in a million runs passes every normal test. It also depends on the machine: more cores, a different load, an optimised build or an extra `printf` change the timing. Hence the modules test twice — under ThreadSanitizer, which reports the data race whenever the unordered accesses *happen*, whether or not they collide in that run, and optimised at full speed, where broken invariants become visible.

#### In SWEB

SWEB is preemptive: the timer interrupt can switch threads at almost any instruction, even in kernel code (`syscallHandler` enables interrupts before calling `Syscall::syscallException`). After A1, several threads of one process can be in the kernel at the same time, touching the same process structures — the thread list, the address space, the file descriptors. Every such structure needs a lock, and every “check, then act” in your `pthread_join` (“is the thread finished? if not, sleep”) is a critical section. The A1 plan says it bluntly: *racess appear on the tenth run, not the first.*

#### Key points

- `x++` is load–modify–store; interleavings lose updates.
- Check-then-act is a race even without arithmetic; in the kernel it is TOCTOU.
- Find critical sections through invariants; fix them by locking, atomics, or not sharing.
- Data race = unordered conflicting accesses (undefined behaviour); race condition = timing-dependent correctness.
- Only happens-before edges (locks, create/join, atomics) guarantee visibility; `volatile` is not synchronisation.